

# Laporan Praktikum Kontrol Cerdas

Nama : Arjuna Kurniawan Kusuma Putra

NIM : 224308004

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/kakjuna>

Student Lab Assistant : Rizky Putri Ramadhani (214308092)

## 1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

## 2. Tujuan Percobaan

Tujuan dari percobaan praktikum Canny Edge Detection & Lane Detection with Instance Segmentation yaitu:

1. Memahami konsep Canny Edge Detection sebagai metode dasar deteksi
2. Menggabungkan metode Canny Edge Detection dengan Instance Segmentation untuk meningkatkan deteksi jalur.
3. Menggunakan Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection).

## 3. Landasan Teori

Canny Edge Detection atau deteksi tepi adalah langkah penting dalam proses pengolahan citra yang bertujuan untuk mengidentifikasi batas-batas atau kontur objek dalam gambar, seperti Sobel, Prewitt, dan Roberts telah lama digunakan dalam dunia pengolahan citra. Namun, metode Canny Edge Detection dianggap lebih unggul karena mampu mendeteksi tepi dengan akurasi yang lebih tinggi dan menghasilkan gambar dengan tingkat kebisingan yang rendah (Luh Putu Risma Noviana dkk., 2024). Keunggulan Canny Edge Detection dibandingkan dengan deteksi tepi lainnya yaitu good detection, memaksimalkan Signal to Noise Ration (SNR) agar semua tepi dapat terdeteksi dengan baik, kemudian good location, untuk meminimalkan jarak deteksi tepi yang sebenarnya dengan tepi yang dihasilkan melalui pemrosesan, sehingga lokasi tepi terdeteksi menyerupai tepi secara nyata (semakin besar nilai Loc, maka semakin besar kualitas deteksi yang dimiliki), selanjutnya ada one respon to single edge, untuk menghasilkan tepi tunggal /tidak memberikan tepi yang bukan tepi sebenarnya (Perangin-angin dkk., 2019). Pada praktikum ini juga menggunakan proses Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection). Instance Segmentation atau proses anotasi merupakan proses pemberian label pada suatu gambar, biasanya menggunakan tools Roboflow (Wulanningrum dkk., 2024). Pelatihan datanya menggunakan kurang lebih 2000 foto rel kereta api dengan berbagai macam track, ada yang

lurus, berbelok, single track, double track, dan lain-lain. Setiap epoch melibatkan pemasukan dataset ke dalam model, penyesuaian parameter model, dan pembaruan bobot untuk mengoptimalkan kinerja model dalam mendeteksi dan segmentasi rel kereta api dengan akurat. Proses iteratif ini bertujuan untuk meminimalkan perbedaan antara rel kereta api yang diprediksi dengan objek lain dan anotasi kebenaran dasar dalam dataset pelatihan. Teknik augmentasi data diterapkan selama pelatihan untuk meningkatkan kemampuan generalisasi model (Husain dkk., 2024). Algoritma ini tahan terhadap oklusi yang kompleks, sehingga sangat berguna untuk adaptasi terhadap lingkungan AV. Algoritma ini telah digabungkan oleh arsitektur jaringan klasifikasi seperti ENet, Segnet, dan Resnet38 yang dibuat khusus untuk identifikasi gambar. Hasilnya adalah arsitektur Resnet38 memiliki akurasi rata-rata yang paling tinggi, namun membutuhkan memori yang lebih besar (Darwin & Saputro, 2021).

## 4. Analisis dan Diskusi

- Analisis

A. Deteksi jalur (lane detection) dengan Instance Segmentation umumnya jauh lebih baik dibandingkan dengan metode Canny Edge Detection dalam skenario yang kompleks. Hal ini disebabkan oleh perbedaan mendasar dalam pendekatan kedua metode tersebut.

- 1) Canny Edge Detection: Terlalu Sensitif terhadap Noise, Tidak Memahami Konteks, Sulit Menangani Kondisi Kompleks.

- 2) Instance Segmentation: Pemahaman Konteks, Robust terhadap Noise dan Pencahayaan, Kemampuan Multiklas dan Multi-Objek, Penanganan Situasi Kompleks.

B. Mengombinasikan Canny Edge Detection dan Instance Segmentation dapat meningkatkan akurasi deteksi jalur dalam beberapa skenario, terutama untuk menciptakan solusi yang lebih robust dan efisien. Kombinasi ini berpotensi menggabungkan keunggulan masing-masing metode dengan cara yang saling melengkapi. Kombinasi Canny Edge Detection dan Instance Segmentation dapat meningkatkan akurasi dan efisiensi, terutama dengan pendekatan berbasis ROI. Namun, efektivitasnya bergantung pada bagaimana metode ini diintegrasikan. Pada kasus dengan jalur yang sangat kompleks (misalnya persimpangan rumit), Instance Segmentation tetap menjadi tulang punggung utama, sedangkan Canny lebih berguna untuk mempercepat proses dan meningkatkan akurasi pada detail halus. Untuk implementasi yang optimal, penting untuk melakukan eksperimen, tuning parameter, dan validasi di berbagai kondisi jalan agar sistem dapat beradaptasi dengan baik di lingkungan dunia nyata.

C. Parameter dalam Canny Edge Detection sangat memengaruhi hasil deteksi tepi. Dua parameter utama yang biasanya diubah adalah threshold rendah dan threshold tinggi, yang menentukan sensitivitas deteksi tepi. Berikut adalah dampak dari perubahan parameter tersebut:

- 1) Lower Threshold ( $T_1$ ): Digunakan untuk mendeteksi tepi dengan intensitas gradien yang lebih rendah. Pixel dengan gradien di bawah nilai ini akan diabaikan.

- 2) Upper Threshold ( $T_2$ ): Digunakan untuk mendeteksi tepi yang lebih jelas. Pixel dengan intensitas di atas nilai ini dianggap sebagai tepi kuat (strong edges).

- 3) Pixel dengan nilai antara kedua threshold akan dipertahankan hanya jika mereka terhubung dengan tepi yang kuat (hysteresis thresholding).

- Diskusi

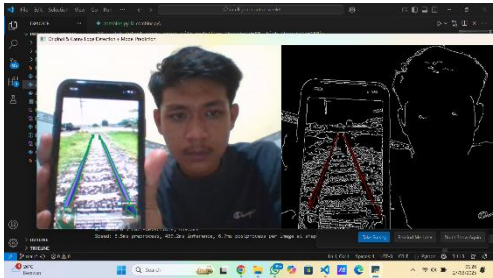
- A. Canny Edge Detection lebih baik digunakan dalam situasi tertentu dibandingkan Instance Segmentation, terutama ketika ada kendala komputasi, lingkungan yang sederhana, atau kebutuhan deteksi cepat dengan resource yang terbatas. Namun, ada batasan di mana Canny mulai gagal, dan Instance Segmentation lebih baik digunakan: Kondisi Jalan yang Kompleks, Kehadiran Banyak Gangguan, Kebutuhan Deteksi Multiklas.
- B. Untuk meningkatkan deteksi jalur menggunakan YOLOv8-seg, Anda dapat melakukan tuning parameter dengan menyesuaikan beberapa aspek penting. Pertama, optimalkan learning rate agar model bisa belajar lebih efektif tanpa overfitting atau underfitting, misalnya dengan mencoba learning rate scheduler seperti Cosine Annealing. Kedua, atur batch size sesuai kapasitas memori GPU; gunakan batch besar untuk stabilitas pelatihan atau gradient accumulation jika memori terbatas. Selanjutnya, lakukan optimasi anchor box dengan algoritme k-means clustering untuk memastikan ukuran dan bentuk anchor sesuai dengan dimensi jalur yang dideteksi. Tingkatkan resolusi gambar input jika jalur tipis sulit terdeteksi, meskipun ini akan menambah waktu inferensi. Selain itu, terapkan data augmentation seperti flip horizontal, adjustment brightness, dan perspective transformation agar model mampu beradaptasi dengan kondisi lingkungan yang beragam. Untuk meningkatkan hasil segmentasi, sesuaikan weight pada loss function sehingga fokus model lebih optimal pada jalur utama dan mengabaikan noise di latar belakang. Terakhir, gunakan fine-tuning dengan transfer learning dari model pretrained, sehingga pelatihan lebih cepat dan akurasi pada dataset spesifik bisa meningkat.
- C. Penerapan deteksi jalur berbasis YOLOv8-seg dalam sistem navigasi kereta otomatis memberikan sejumlah manfaat penting. Model ini mampu mendeteksi dan melakukan segmentasi jalur rel secara akurat dalam waktu nyata (real-time), yang berfungsi untuk memastikan kereta tetap berada di lintasan yang benar. Selain itu, sistem ini bisa mengenali percabangan rel, mendeteksi rintangan seperti benda jatuh atau pejalan kaki, dan menyesuaikan kecepatan kereta secara otomatis guna meningkatkan keamanan. Dengan menggunakan data augmentasi dan tuning hyperparameter yang tepat (seperti learning rate, batch size, dan anchor boxes), model YOLOv8-seg dapat dilatih untuk beradaptasi pada berbagai kondisi lingkungan, termasuk perubahan pencahayaan (siang/malam) dan cuaca ekstrem (hujan, kabut, atau salju). Setelah pelatihan, hasil deteksi rel dan rintangan dioptimalkan melalui post-processing, seperti Non-Maximum Suppression (NMS) untuk mengurangi deteksi berlebihan dan morphological operations agar segmentasi rel lebih halus. Langkah ini memastikan bahwa navigasi otomatis berjalan dengan lebih akurat, stabil, dan mampu mengambil keputusan cepat, misalnya untuk berhenti jika ada rintangan di depan rel. Pengujian lapangan diperlukan untuk memvalidasi performa model dalam kondisi dunia nyata, dengan fokus pada metrik seperti akurasi segmentasi, recall, precision, dan latency.

## 5. Assignment

Berdasarkan praktikum keenam yang telah dilakukan dari kode program ini adalah membuat sebuah sistem yang dapat melakukan deteksi tepi menggunakan metode Canny Edge Detection dan mengombinasikannya dengan deteksi objek atau segmentasi menggunakan model YOLO dari Ultralytics untuk menyoroti objek yang terdeteksi dalam sebuah video yang diambil secara real-time dari kamera. Program dimulai dengan memuat model YOLO dari path yang telah ditentukan menggunakan `YOLO(MODEL_PATH)`, memastikan bahwa model tersedia sebelum melanjutkan ke

pemrosesan video menggunakan OpenCV. Setelah kamera diakses melalui `cv2.VideoCapture(0)`, setiap frame yang ditangkap dikonversi ke skala abu-abu dengan `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)` agar dapat dilakukan deteksi tepi menggunakan metode Canny Edge Detection, yang menghasilkan gambar biner dengan tepi putih pada latar belakang hitam melalui `cv2.Canny(gray, low_threshold, high_threshold)`. Secara paralel, frame asli dikonversi ke format RGB dengan `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`, kemudian diproses oleh model YOLO untuk melakukan prediksi objek atau segmentasi, di mana hasil prediksi pertama disimpan dalam `predictions = results[0]`. Jika model menggunakan segmentasi, maka program akan menggambar area objek menggunakan poligon biru transparan dengan `cv2.fillPoly(overlay, [mask], (255, 0, 0))`, sedangkan jika model hanya mendeteksi objek dalam bentuk bounding box, maka objek akan dilingkari dengan kotak hijau menggunakan `cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)`, serta label objek akan ditampilkan jika tersedia menggunakan `cv2.putText()`. Selanjutnya, masking digunakan untuk membedakan tepi objek yang terdeteksi oleh YOLO dengan tepi yang dideteksi oleh metode Canny, di mana tepi dalam area objek yang dikenali YOLO akan diwarnai merah, sementara tepi lainnya tetap berwarna putih dengan `edges_bgr[np.where((mask_model == 255) & (edges == 255))] = [0, 0, 255]`. Kedua hasil, yaitu frame dengan anotasi YOLO dan hasil deteksi tepi, ditampilkan secara berdampingan menggunakan `np.hstack()` untuk memberikan tampilan visual yang lebih jelas mengenai perbedaan antara tepi umum dan tepi dari objek yang dikenali. Program berjalan dalam loop hingga pengguna menekan tombol 'q', yang akan memicu `cv2.waitKey(1) & 0xFF == ord('q')` untuk keluar, setelah itu kamera akan dilepaskan dengan `cap.release()` dan semua jendela yang terbuka akan ditutup dengan `cv2.destroyAllWindows()` untuk mencegah kebocoran memori. Dengan kombinasi dari metode deteksi tepi tradisional dan teknologi deep learning untuk segmentasi atau deteksi objek, program ini dapat digunakan untuk berbagai aplikasi seperti deteksi rel, pengawasan lalu lintas, dan lainnya.

## 6. Data dan Output Hasil Pengamatan

| No | Variabel             | Hasil Pengamatan                                                                    |                                                                                       |
|----|----------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1  | Canny Edge Detection |  |  |
| 2  | Lane Detection       |  |  |
| 3  | Combined             |  |  |

## 7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

1. Teknik masking digunakan untuk membedakan tepi objek yang terdeteksi YOLO dengan tepi umum dari hasil Canny Edge Detection, memungkinkan visualisasi yang lebih jelas.
2. Program berhasil mengombinasikan metode Canny Edge Detection dengan model YOLO untuk mendeteksi tepi sekaligus mengenali dan menyensor objek dalam video real-time.
3. Program dapat berjalan dalam loop hingga pengguna menekan tombol keluar, dengan pengelolaan memori yang baik untuk menghindari kebocoran sumber daya

## 8. Saran

Untuk meningkatkan performa dan akurasi, program dapat dioptimalkan dengan penggunaan GPU agar prediksi YOLO berjalan lebih cepat dan responsif. Selain itu, preprocessing tambahan seperti Gaussian Blur sebelum Canny Edge Detection dapat membantu mengurangi noise yang tidak diperlukan. Untuk meningkatkan keterbacaan visual, transparansi overlay warna dapat disesuaikan agar objek yang terdeteksi lebih jelas tanpa menutupi detail penting dalam gambar. Menambahkan fitur penyimpanan hasil deteksi dalam bentuk gambar atau video bisa menjadi nilai tambah untuk analisis lebih lanjut di berbagai aplikasi dunia nyata.

## 9. Daftar Pustaka

Husain, B. H., Osawa, T., Astiti, S. P. C., Frianto, D., Nandika, M. R., & Agustine, D. (2024). Deteksi Lubang Jalan Secara Otomatis dari Rekaman Drone Menggunakan Model Machine Learning Berbasis YOLOv5 Instance Segmentation di Kota Pekanbaru, Provinsi Riau, Indonesia. *Jurnal Zona*, 8(2), 137–146.

Luh Putu Risma Noviana, I Putu Eka Indrawan, & Gde Iwan Setiawan. (2024). ANALYSIS OF CANNY EDGE DETECTION METHOD FOR FACIAL RECOGNITION IN DIGITAL IMAGE PROCESSING. *Jurnal Manajemen dan Teknologi Informasi*, 15(2), 29–34.